



Asistente LLM de Meteorología Aeronáutica

METAR / TAF vía LangChain + FastAPI

Documento Técnico — Componentes y Arquitectura del Sistema

Proyecto: Aviation Weather Assistant | Septiembre 2026

1. Introducción y propósito del proyecto

Aviation Weather Assistant es un backend construido con **FastAPI** que ofrece una interfaz de chat impulsada por un modelo de lenguaje (LLM) para consultar información meteorológica aeronáutica — reportes **METAR** (condición actual) y **TAF** (pronóstico de aeródromo). Un usuario típico, por ejemplo un estudiante de vuelo, formula una pregunta en lenguaje natural como «dame el METAR del aeropuerto de El Paso». El agente LLM resuelve el nombre del aeropuerto a su código ICAO, invoca una herramienta («tool») de LangChain que consulta la API pública de **aviationweather.gov**, y finalmente decodifica el texto crudo del reporte (rawOb / rawTAF) en una explicación clara en lenguaje natural, acompañada del código crudo (raw) original.

🔗 *La resolución de nombre de aeropuerto → código ICAO y la decodificación del reporte METAR/TAF son responsabilidad exclusiva del LLM: el sistema deliberadamente no incluye un parser manual ni una tabla de aeropuertos local, para no duplicar una capacidad que el modelo ya cubre bien.*

2. Stack tecnológico

El proyecto combina un backend Python orientado a agentes de IA con una interfaz web moderna, apoyándose en servicios de infraestructura para persistencia, resiliencia y observabilidad.

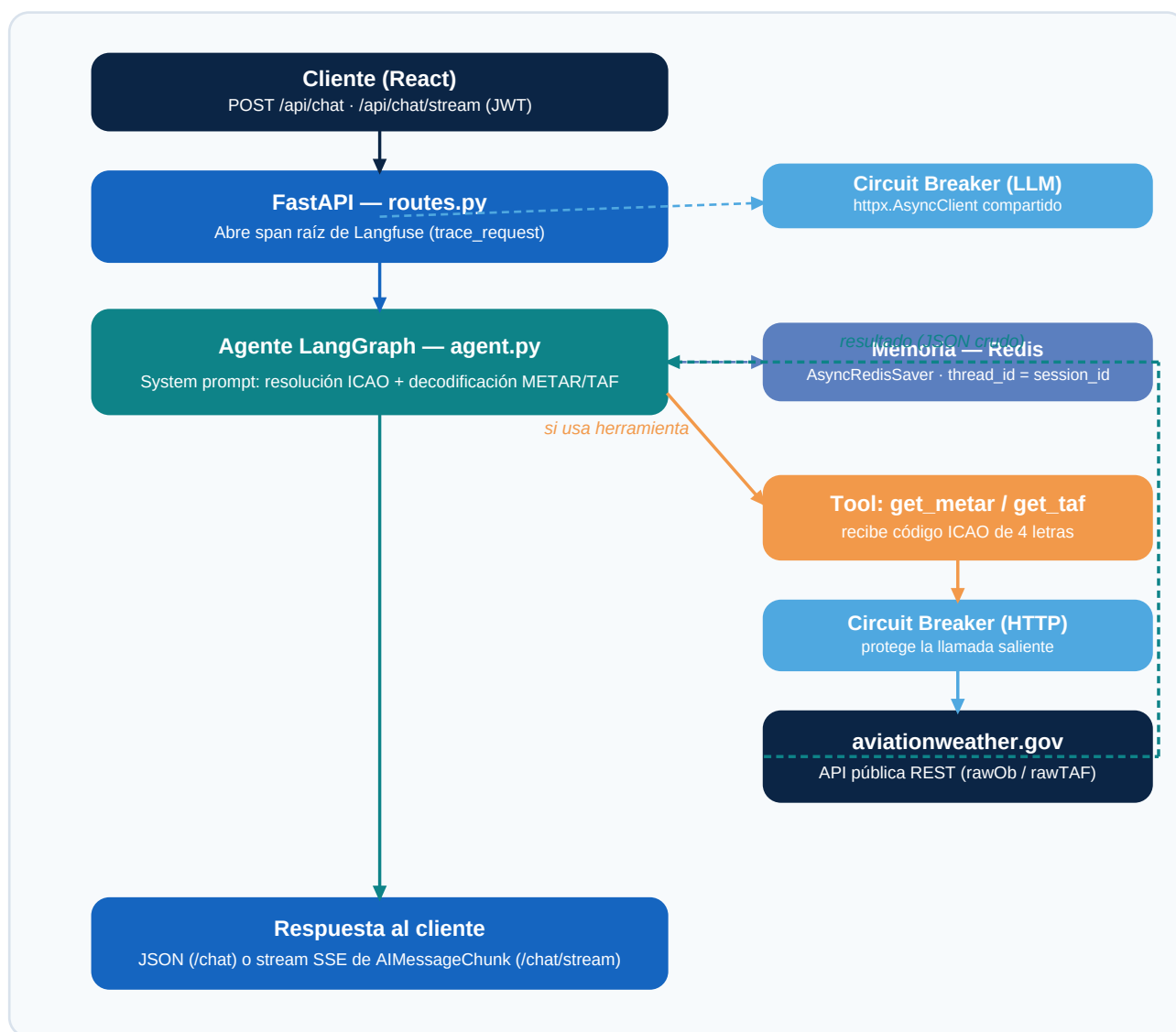
Componente	Tecnología	Función
API HTTP	FastAPI	Expone los endpoints REST/SSE del chat y gestiona el ciclo de vida de la app.
Orquestación LLM	LangChain (langchain, langchain-openai, langchain-community)	Agente con uso de herramientas («tool-calling») y memoria conversacional.
Proveedor LLM	OpenAI (vía langchain-openai / ChatOpenAI)	Modelo generativo configurable por variables de entorno.
Cliente HTTP	httpx	Llamadas salientes asíncronas a aviationweather.gov.
Configuración	pydantic-settings	Carga de variables de entorno y archivo .env.
Memoria de conversación	Redis 8+ / langgraph-checkpoint-redis	Checkpoint persistente entre procesos, indexado por thread_id = session_id.
Autenticación	JWT (python-jose) + bcrypt	Emisión y validación de tokens de acceso vía OAuth2PasswordBearer.
Observabilidad	Langfuse (opcional)	Trazabilidad por request de llamadas LLM y herramientas.
Pruebas	pytest + respx	Pruebas automatizadas; las llamadas HTTP externas siempre se simulan (mock).
Contenerización	Docker / docker compose	Empaquetado y orquestación de todos los servicios.
Frontend	React + TypeScript (Vite)	Cliente de chat estilo chatgpt.com, servido por nginx en producción.

3. Arquitectura del sistema

La arquitectura sigue un patrón de **agente con herramientas (tool-calling agent)**: el backend expone endpoints de chat que delegan en un grafo LangGraph construido sobre LangChain 1.x. El agente decide, en cada turno, si responde directamente o invoca una herramienta externa antes de generar la respuesta final.

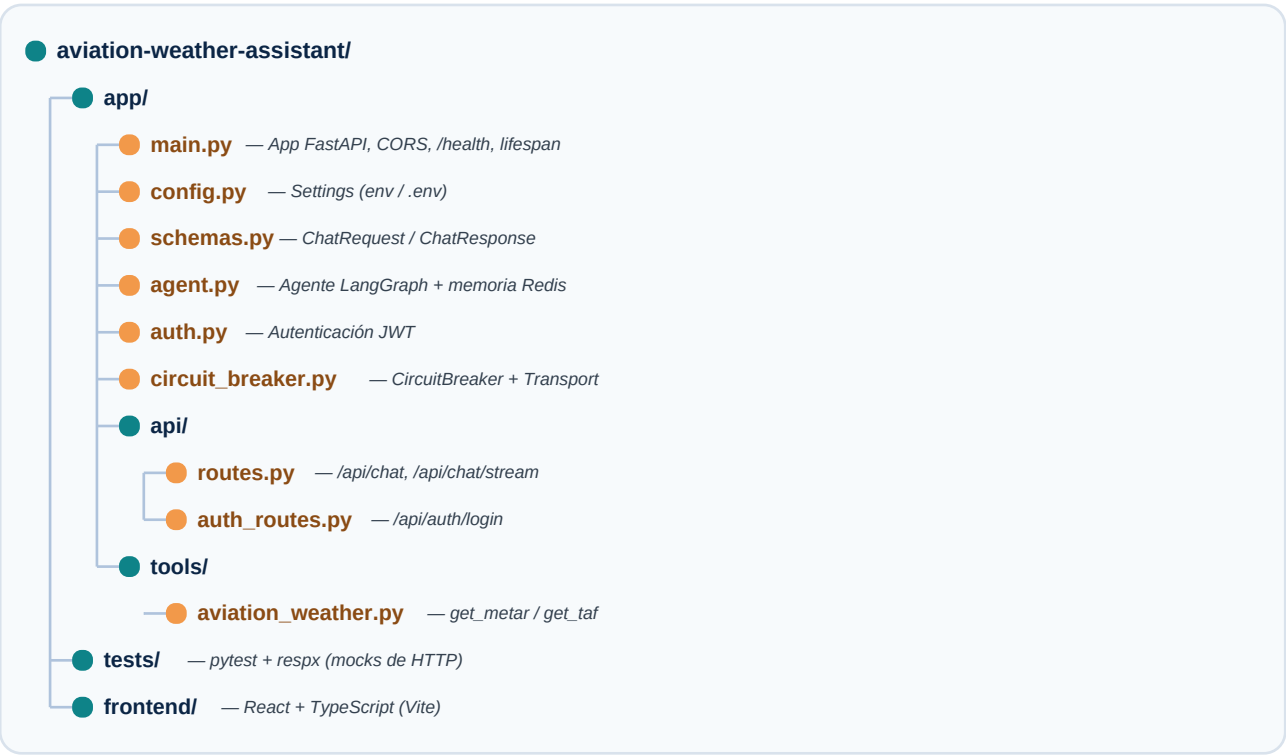
3.1 Diagrama de flujo de una petición

El siguiente diagrama muestra el recorrido completo de una petición de chat: desde el cliente React, pasando por FastAPI y el agente LangGraph, hasta la posible invocación de la herramienta meteorológica y el retorno de la respuesta.



3.2 Estructura de directorios

Árbol de directorios y archivos principales del backend, con una breve nota sobre la responsabilidad de cada módulo.



4. Componentes principales

4.1 Agente conversacional (app/agent.py)

Construye el grafo del agente con `create_agent` de LangChain 1.x. La memoria por sesión se implementa mediante un checkpointer `AsyncRedisSaver` de LangGraph, usando el `thread_id` igual al `session_id` de la conversación. El checkpointer se inicializa una única vez en el manejador *lifespan* de `main.py` (no por cada request), y `build_agent()` lanza un error si se invoca antes de completar el arranque de la aplicación.

4.2 Herramientas LangChain (app/tools/aviation_weather.py)

Expone las funciones `get_metar` y `get_taf`, decoradas como `@tool`, que reciben únicamente un código ICAO de 4 letras y devuelven el JSON crudo de `aviationweather.gov` tal cual. La interpretación del contenido se delega por completo al modelo mediante el `system prompt`.

4.3 Memoria persistente en Redis

La conversación sobrevive a reinicios del proceso y se comparte entre workers/instancias que apunten al mismo Redis. Requiere **Redis 8+** (o Redis Stack), ya que `langgraph-checkpoint-redis` indexa los checkpoints con `RedisSearch/RedisJSON` — un Redis estándar sin esos módulos no funciona. Se configura mediante la variable `REDIS_URL`.

4.4 Streaming de respuestas (SSE)

El endpoint `/api/chat/stream` usa `agent.astream(..., stream_mode="messages")` y reenvía cada fragmento como una línea SSE `data:.` Es crítico filtrar únicamente instancias de `AIMessageChunk`, ya que `stream_mode="messages"` también emite fragmentos de `ToolMessage` (el JSON crudo de la herramienta), que también tienen contenido no vacío. Sin ese filtro, el JSON crudo se filtraría dentro del flujo de tokens antes de la respuesta real del modelo — un bug real detectado al probar el frontend en modo streaming con una consulta que dispara una herramienta.

4.5 Circuit breakers (app/circuit_breaker.py)

Protegen las dos dependencias externas críticas para que una caída falle rápido en lugar de acumular peticiones colgadas:

- Uno envuelve las llamadas a las herramientas de `aviationweather.gov`.
- Otro envuelve todas las llamadas al proveedor LLM, a través de un `httpx.AsyncClient` compartido pasado como `ChatOpenAI(http_async_client=...)` — cubre `ainvoke/astream`/llamadas a herramientas, sin importar qué ruta de LangGraph las dispare.

El estado del breaker se expone en `GET /health` y ante un corte, se traduce en HTTP 503 (o un evento SSE `{"error": ...}` en streaming). Es configurable vía `CIRCUIT_BREAKER_FAILURE_THRESHOLD` / `CIRCUIT_BREAKER_RESET_TIMEOUT`.

4.6 Autenticación (app/auth.py)

Implementa autenticación basada en **JWT** para un único usuario de demostración (credenciales definidas en `config.py`). Las contraseñas se verifican con `bcrypt`, y los tokens se emiten y validan con `python-jose` a través del esquema `OAuth2PasswordBearer` apuntando a `/api/auth/login`. No está diseñado para escalar

a múltiples usuarios; requeriría sustituirse por un almacén de usuarios real.

4.7 Trazabilidad con Langfuse (app/agent.py: trace_request)

Abre un *span* raíz de Langfuse por cada petición HTTP, desde la llegada al manejador de ruta hasta la respuesta, usando `Langfuse.start_as_current_observation` con un `trace_id` generado explícitamente. El `CallbackHandler` de `LangChain` se construye con un `trace_context` explícito en lugar de depender del contexto OTel ambiental, ya que la ejecución asíncrona de `LangGraph` no lo propaga de forma confiable (limitación conocida de `langfuse/langgraph`). El sistema no realiza ninguna operación (no-op) si Langfuse no está configurado.

5. Decisiones de diseño clave

Decisión	Justificación
Resolución ICAO por el LLM	El modelo ya resuelve bien nombres de aeropuertos a códigos ICAO para aeropuertos principales; una tabla local añadiría un dataset a mantener sin necesidad real.
Decodificación METAR/TAF por el LLM	El system prompt instruye al modelo a decodificar el reporte campo por campo; se evita duplicar esa lógica con un parser manual.
Memoria en Redis, no en memoria de proceso	Permite persistencia entre reinicios y consistencia entre múltiples workers/instancias que compartan el mismo Redis.
Circuit breakers en ambas dependencias externas	Aviationweather.gov y el proveedor LLM son puntos de fallo externos; sin breaker, una caída podría acumular peticiones colgadas indefinidamente.
Filtrado estricto de AIMessageChunk en streaming	Evita que el JSON crudo de las tool calls se filtre en el flujo de texto entregado al usuario final.
Un único span raíz de Langfuse por request	Permite correlacionar la latencia de cada llamada LLM y de cada tool call bajo un mismo trace_id, facilitando el diagnóstico de rendimiento.
Frontend desacoplado (SPA pura)	El frontend React no es servido por FastAPI; siempre llama a rutas /api/* del mismo origen, proxied por Vite en desarrollo y por nginx en producción, evitando URLs de backend embebidas.

6. Ejecución y despliegue

6.1 Entorno local

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
copy .env.example .env # configurar OPENAI_API_KEY
uvicorn app.main:app --reload
pytest
```

6.2 Frontend

```
cd frontend
npm install
npm run dev # proxy de /api hacia localhost:8000
```

6.3 Contenerizado (Docker Compose)

El comando `docker compose up --build` levanta el backend, el servicio Redis 8 y el frontend (servido por nginx) en `localhost:3000`, leyendo la configuración desde el archivo `.env`.

6.4 Convenciones del proyecto

- Todo I/O (llamadas HTTP, invocación del agente) es asíncrono; las nuevas rutas y herramientas deben mantenerse `async`.
- Las pruebas nunca deben golpear la API real de `aviationweather.gov`; se simulan con `respx`.
- Las nuevas herramientas LangChain se ubican en `app/tools/`, un módulo por API externa, y se registran en la lista `_TOOLS` de `app/agent.py`.
- Toda nueva llamada saliente a un servicio externo del que dependa la app en tiempo de petición debe pasar por un `CircuitBreaker`, igual que las herramientas de `aviationweather.gov` y el cliente LLM.

Documento generado a partir del estado actual del repositorio Aviation Weather Assistant (rama main).